

# Informática Culinaria y Máquinas de Estado

Paradigmas de Ejecución de Datos, Esquemas de Ingredientes y Ontologías Robóticas

**Autor:** Félix Martínez    **Dominio:** Informática Culinaria & Web Semántica    **Estado:** Arquitectura Avanzada (v2.0)

La gestión de datos culinarios ha operado históricamente bajo un paradigma de retención documental, diseñado fundamentalmente para la lectura humana. Los sistemas actuales de gestión de recetas tratan las instrucciones como historia estática, cadenas de texto plano que carecen de semántica mecánica. Sin embargo, la convergencia de la web semántica, los motores de simulación de videojuegos y la robótica autónoma exige un cambio drástico. El emergente campo interdisciplinario de la Informática Culinaria y de Alimentos requiere que una receta deje de ser un documento pasivo para convertirse en un **script de ejecución determinista**, capaz de ser validado, transformado y ejecutado por una máquina de estados o un agente robótico.

**Definición del Dominio:** La *Informática de Alimentos (Food Informatics)* adquiere, organiza, analiza e interpreta datos culinarios a través de la intersección de la ciencia alimentaria y la computación. A diferencia de la Computación Alimentaria (orientada a tareas aisladas como visión por computador o recomendación dietética), la Informática Culinaria subraya la representación e integración de información heterogénea en todas las capas del ecosistema.

## 1. Evolución Histórica de los Formatos Culinarios Tradicionales

A lo largo de las décadas, la ingeniería de software ha intentado estructurar la variabilidad inherente del lenguaje natural en la cocina, con diferentes compensaciones entre legibilidad y poder de cómputo:

| Formato de Datos     | Representación de Ingredientes | Comprensión Humana        | Ecosistema / Parsers   | Propósito Arquitectónico         |
|----------------------|--------------------------------|---------------------------|------------------------|----------------------------------|
| MealMaster           | Columnas de ancho fijo         | Limitada y rígida         | Sistemas heredados     | Archivo de texto plano pre-web   |
| RecipeML             | Nodos XML (<qty>, <item>)      | Nula (Verbosidad extrema) | Obsoleto               | Bases de datos estructuradas     |
| Schema.org (JSON-LD) | Cadenas de texto en matrices   | Nula (Para navegadores)   | Universal (JSON)       | Optimización SEO y Rich Snippets |
| Cooklang (DSL)       | Nodos estructurados en línea   | Excelente (Prosa natural) | Activo (15+ lenguajes) | Autoría humana y CLI/ Robótica   |

En la especificación Schema.org/Recipe, atributos clave como `recipeIngredient` y `recipeInstructions` se definen como matrices unidimensionales de cadenas de texto. Una declaración como *"3 dientes de ajo, picados finamente"* es legible para un rastreador web, pero para un sistema de simulación carece de significado estructurado. Esto produce el denominado *"problema del viaje de ida y vuelta con pérdidas"* (lossy round-trip problem): al usar JSON-LD como fuente primaria, la estructura subyacente de la preparación se pierde de manera irreversible.

## 2. El Paradigma de Código Abierto: OpenRecipe y Arquitectura ECS

El marco propuesto por **OpenRecipe** y su extensión de ejecución **Open Culinary Runtime (OCR)** adoptan el patrón arquitectónico de **Entidad-Componente-Sistema (ECS)**, tradicional en motores de videojuegos y simulaciones físicas deterministas. El ECS disocia los datos de la lógica de visualización y ejecución:

- **Entidades (El Diccionario):** Encapsulan las capacidades inherentes, estados posibles y propiedades termofísicas de ingredientes y herramientas (ej. conductividad térmica, densidad, estado de agregación).
- **Recetas (El Script de Ejecución):** Inventarios de requisitos estrictos y secuencias inmutables de acciones mecánicas y térmicas.
- **Aplicaciones (Los Intérpretes):** Motores de ejecución que interpretan el script (ya sea la interfaz gráfica React en web o el motor de físicas Unity para un brazo robótico).

**Modelado de Transformaciones Transaccionales:** Bajo ECS, un ingrediente no es un texto sino un nodo lógico. La separación de la clara y la yema de un huevo destruye computacionalmente la entidad compuesta matriz e instancia dos entidades hijas independientes, cada una con propiedades térmicas e higiénicas diferenciadas para las etapas subsecuentes.

## 3. Lenguajes de Dominio Específico: El Ecosistema Cooklang

Cooklang permite redactar recetas en archivos de texto plano (.cook) manteniendo soporte para marcado semántico accesible:

```
Añada @sal al agua.  
Tamice @harina de trigo{500%g}.  
Mézcle usando un #batidor de varillas{ }.  
Hierva la ~pasta{10%minutos}.
```

El compilador de referencia (**cooklang-rs**, escrito en Rust) ejecuta una canalización formal:

1. **Lexing:** Escaneo del archivo UTF-8 y tokenización.
2. **Parsing & AST:** Construcción del Árbol de Sintaxis Abstracta (ScalableRecipe) vincular posicionalmente cada ingrediente con el paso exacto.
3. **Análisis Semántico:** Resolución de referencias relativas entre recetas anidadas.
4. **Generación / Escalado:** Emisión de ScaledRecipe aplicando conversión de unidades y reglas de proporción.

## 4. Inferencia Semántica Dinámica y Grafos de Conocimiento

Para prevenir fallos ante ingredientes no reconocidos en el diccionario canónico, los motores avanzados implementan **Inferencia Semántica de Capacidades**. Ante la frase "*Pelar la @fruta de la pasión, picarla y saltearla*", el analizador deduce dinámicamente las propiedades mecánicas del ente desconocido en tiempo de ejecución (**isPeelable: true, isChoppable: true, isFryable: true**).

Finalmente, mediante ontologías de la Web Semántica como **FoodOn** (respaldada por OBO Foundry), estas máquinas de estado integran grafos de conocimiento que conectan perfiles organolépticos, nutricionales y protocolos de seguridad alimentaria en tiempo real.